

Introduction

Every interview eventually circles back to the same question in disguise: "you have a fixed collection of values — how fast can you check membership?" LeetCode 3718, Smallest Missing Multiple of K, is a clean, small-input problem that quietly tests exactly that instinct. It looks simple on the surface, but the naive approach hides a quadratic trap that a lot of candidates walk straight into.

What makes this problem worth learning isn't the problem itself — it's the pattern underneath it. Once you see it here, you'll recognize it in dozens of other interview questions: whenever you're repeatedly asking "is this value present?" against a collection that doesn't change, a hash set turns that question from an $O(n)$ scan into an $O(1)$ lookup. That single idea is the difference between a solution that scales and one that times out on a slightly bigger input.

Problem Overview

You're given an array of integers, `nums`, and a positive integer `k`. Think of `k`'s multiples as a numbered list of tickets: `k`, `2k`, `3k`, `4k`, and so on, forever. Some of those ticket numbers have already been "claimed" — they show up somewhere in `nums`. Your job is to find the smallest multiple of `k` that has *not* been claimed.

Formally: return the smallest positive integer `x` such that `x` is a multiple of `k` and `x` does not appear in `nums`.

It's worth being precise about what this problem is *not*. It's easy to confuse it with the classic "smallest missing positive integer" problem, where you search every integer starting from 1 and bound your search at `n + 1`. This problem is narrower — you only ever need to test integers that are multiples of `k`. That narrower search space is exactly what makes the hash set approach so effective here.

Example

Let's walk through a concrete case. Suppose:

```
nums = [8, 2, 3, 4, 6]
k = 2
```

The multiples of `k` we care about, in order, are 2, 4, 6, 8, 10, 12...

- Is 2 in `nums` ? Yes — claimed.
- Is 4 in `nums` ? Yes — claimed.
- Is 6 in `nums` ? Yes — claimed.
- Is 8 in `nums` ? Yes — claimed.
- Is 10 in `nums` ? No — this is our answer.

Every multiple up to 8 happened to be blocked by some value in the array, but 10 was never claimed, so `10` is the smallest missing multiple of `2`.

Notice something important here: the array contains five numbers, and it took exactly five checks (2 through 10) before we hit a free one. That's not a coincidence — it's the core guarantee the whole algorithm leans on.

Intuition

Here's how an experienced engineer would think through this before writing a line of code.

First, notice the shape of the search: you're not searching over all integers, you're searching over a specific, evenly spaced sequence — `k`, `2k`, `3k`, `...`. That already tells you the answer exists and isn't infinite, but it doesn't yet tell you how *far* you might have to search.

Second, ask: how many of those multiples can possibly be blocked? Every value in `nums` can block at most one multiple of `k` — a given number is only equal to one specific multiple of `k` (if it's a multiple of `k` at all). With `n` numbers in the array, at most `n` of the multiples `k`, `2k`, `3k`, `...`, `(n+1)k` can be claimed. That means the `(n+1)`th multiple is *guaranteed* to be free, even in the worst case where every single array value happens to be a distinct multiple of `k`. This bounds the walk to at most `n + 1` steps — small and predictable.

Third, ask what the expensive part of a naive approach actually is. It's not the walk over multiples — that's at most `n + 1` steps. It's the *membership check* at each step. If you rescan the whole array every time you ask "is this multiple in `nums` ?", you're paying `O(n)` per check, `O(n)` checks, and you land on `O(n2)` overall.

The fix falls out naturally once you frame it that way: don't rescan. Load every value from `nums` into a hash set once, up front. After that, "is this number claimed?" becomes an `O(1)` lookup, and the total cost collapses to the size of the walk itself — linear time.

Brute Force Solution

Idea: For each candidate multiple of `k`, scan the entire array to check whether it's present.

Algorithm: 1. Start with `multiple = k`. 2. Loop through `nums` to check if `multiple` is present. 3. If it is, increment `multiple` by `k` and repeat. 4. If it isn't, return `multiple`.

Advantages: - Extremely simple to write, easy to reason about correctness. - No extra data structures — $O(1)$ auxiliary space.

Disadvantages: - Every membership check costs $O(n)$, and you may need up to `n + 1` checks, giving $O(n^2)$ time in the worst case. - Wasteful: the array never changes between checks, so re-scanning it every time throws away work you've already done.

Time complexity: $O(n^2)$ — up to `n + 1` candidates, each requiring an $O(n)$ scan. **Space complexity:** $O(1)$ — no extra structures beyond a counter.

Optimal Solution

The optimal approach separates the problem into two clean phases:

Phase 1 — Preprocessing: Convert `nums` into a hash set called `claimed`. This costs $O(n)$ time and $O(n)$ space, but it only happens once.

Phase 2 — Walking: Starting at `multiple = k`, repeatedly check `multiple` in `claimed`. Each check is now $O(1)$ because hash sets support constant-time average-case membership testing. If `multiple` is claimed, add `k` and try again. The first `multiple` not found in `claimed` is the answer.

Step		multiple In claimed?	Action
1	2	Yes	advance
2	4	Yes	advance
3	6	Yes	advance
4	8	Yes	advance
5	10	No	return 10

Why does this terminate quickly? Because — as established in the Intuition section — at most `n` multiples of `k` can be blocked by the `n` values in `nums`. So the loop runs at most `n + 1` times, each iteration doing $O(1)$ work. The total walk cost is $O(n)$, and combined with the $O(n)$ set construction, the whole algorithm is $O(n)$ time.

Python Code

```
def smallest_missing_multiple(nums: list[int], k: int) -> int:
    """Return the smallest positive multiple of k not present in nums."""
    claimed = set(nums)

    multiple = k
    while multiple in claimed:
        multiple += k

    return multiple
```

Code Walkthrough

- `claimed = set(nums)` : Converts the list into a hash set. This is the one-time cost that makes every future lookup $O(1)$.
- `multiple = k` : We start our search at the first multiple of `k`, since 0 isn't a valid answer (the problem asks for a *positive* multiple).
- `while multiple in claimed:` : This is the $O(1)$ membership check, repeated until we find a gap.
- `multiple += k` : Advances to the next multiple in the sequence.
- `return multiple` : Once the loop exits, `multiple` is guaranteed to be the first unclaimed multiple, because we walked the sequence in strictly increasing order.

Dry Run

Using `nums = [8, 2, 3, 4, 6]`, `k = 2`:

1. `claimed = {8, 2, 3, 4, 6}`
2. `multiple = 2` → `2 in claimed` → `True` → `multiple = 4`
3. `4 in claimed` → `True` → `multiple = 6`
4. `6 in claimed` → `True` → `multiple = 8`
5. `8 in claimed` → `True` → `multiple = 10`
6. `10 in claimed` → `False` → loop exits
7. `return 10`

Matches the expected result.

Complexity Analysis

Time: $O(n)$. Building the set costs $O(n)$. The walk costs at most $O(n + 1)$ iterations (proven in the Intuition section), each doing $O(1)$ work. Total: $O(n)$.

Space: $O(n)$, for storing the hash set of claimed values.

These bounds are correct because they depend only on n (the array size), not on the values themselves or on k — the pigeonhole argument (each array value blocks at most one multiple) is what pins the walk length to $n + 1$, regardless of how large the numbers get.

Alternative Solutions

You could sort `nums` first and use binary search to check membership, giving $O(n \log n)$ for the sort and $O(\log n)$ per check — worse than the hash set's $O(1)$ checks, with no upside. A hash set is strictly better here since we don't need order, only membership.

You could also precompute every multiple of k up to some bound and intersect that with `claimed`, but this does more work than necessary — you'd be generating and checking values you might never need if the answer arrives early.

The hash set walk is the simplest and asymptotically best option for this problem's constraints.

Edge Cases

- **Empty array:** `nums = []`. The set is empty, so k itself is immediately unclaimed and returned.
- **All multiples claimed up to a point, one gap far out:** e.g., $k = 5$, every multiple up to 100 present except 95 — the walk correctly reaches 95 without skipping it, since it checks every multiple in order.
- **k itself missing:** If k isn't in `nums` at all, the answer is k on the very first check.
- **Duplicates in nums:** Duplicate values collapse naturally into the set — they don't affect correctness or blow up space beyond $O(\text{unique values})$.
- **Maximum constraints:** With `nums` at 100 elements and k at 100, the walk is bounded to at most 101 steps — trivially fast.

Common Mistakes

1. **Starting the walk at 0 instead of k .** The problem wants a positive multiple; 0 is not valid.

2. **Rescanning the array on every check** instead of building a set first — this silently reintroduces the $O(n^2)$ behavior the set was meant to avoid.
3. **Confusing this with the "smallest missing positive integer" problem** and bounding the search at `n + 1` directly instead of walking multiples of `k`.
4. **Using a list instead of a set for `claimed`**, which makes each `in` check $O(n)$ again, defeating the entire optimization.
5. **Off-by-one errors in the loop**, such as incrementing before checking or checking before assigning the first multiple, which can skip or duplicate a candidate.

Interview Questions

- How would you prove that the algorithm never runs more than `n + 1` iterations?
- What's the time complexity if `nums` could contain up to 10 million elements — does the approach still hold?
- How would this change if you needed the smallest missing multiple of *any* of several different values, not just one `k`?
- Can you solve this without extra space, and what would the tradeoff be?
- How does this problem relate to the classic "first missing positive" problem, and where do the techniques diverge?

Similar Problems

- **First Missing Positive (LeetCode 41):** Same "find the first gap" theme, but over all positive integers rather than multiples of `k`, and typically solved in $O(1)$ space using in-place array tricks.
- **Contains Duplicate (LeetCode 217):** Foundational practice with hash sets for $O(1)$ membership checks.
- **Missing Number (LeetCode 268):** Another missing-value problem, solvable with a set, sum formula, or XOR trick.
- **Two Sum (LeetCode 1):** The canonical "trade a scan for a hash lookup" problem — same underlying pattern as this one.

Key Takeaways

When a problem repeatedly asks "is this value present?" against data that doesn't change, load it into a hash set once and let every subsequent check run in $O(1)$. Here, that one shift took the solution from $O(n^2)$ down to $O(n)$, and the pigeonhole argument — each array value can block at most one multiple

of k — is what guarantees the search terminates quickly. That combination of a bounded search space and constant-time lookups is a pattern worth recognizing on sight.

Watch the Video

For the full walkthrough with the raffle-ticket analogy, a live test run against real examples, and two in-video quizzes to check your understanding, watch the video here: {LINK}

About the Series

This breakdown is part of *Fun with Learning Technology*, a series built around making core computer science and software engineering concepts genuinely click — not just memorized for an interview, but understood well enough to reuse anywhere. Each episode takes a real problem, whether from LeetCode, system design, or everyday engineering work, and builds the intuition from the ground up before ever touching code.

Call To Action

If this explanation helped the problem click, the best way to support the series is to like the video on YouTube and subscribe so future breakdowns land in your feed. For deeper dives, subscribe to the Substack newsletter — it's where the extended write-ups like this one live first. Drop a comment with questions or a problem you'd like covered next, and share this with anyone prepping for interviews.

Quick Review

You need the smallest number $\geq N$ that's both a multiple of K and missing from a given set — what's the pattern?

Walk multiples of K starting from $\text{ceil}(N/K) * K$, skip any in a fast-lookup set, return first hit.

Time complexity of the multiples-walk approach here?

$O((M/K) + S)$ where M is the answer's magnitude and S is set size for lookups — bounded because gaps can't exceed set size.

Space complexity of the solution?

$O(S)$ for the hash set holding the excluded/used values, plus $O(1)$ extra for the walk.

What's the trickiest bug when walking multiples of K?

Starting below N or forgetting to round up to the next multiple — always start at $\text{ceil}(N/K)*K$, not N itself.

Why can't the multiples-walk loop forever?

At most S multiples can be blocked (set size), so within $S+1$ steps you must hit a free multiple.

Brute-force scan vs instant-lookup approach — what's the tradeoff?

Scanning every integer is $O(M)$; converting the exclusion list to a hash set first makes each check $O(1)$, so only K -multiples are visited.

Smallest Missing Multiple of K — free Python cheat sheet